

MOBILE APPLICATION

Mobile Penetration Testing Report

Max it Tunisie · Android application

OVERALL VALIDATED RISK

HIGH

One high-severity and two medium-severity validated findings

FRONT MATTER

Contents

01	Executive Summary	3
02	Engagement Overview	5
03	Methodology and Risk Model	7
04	Validated Findings	9
05	Security Observations	12
06	Pending Validation	16
07	Remediation Roadmap	20

How to read this report

Label	Meaning
Validated finding	A security weakness supported by sufficient technical evidence.
Observation	A condition that increases attack surface or warrants hardening, but is not proven exploitable.
Pending validation	A material test hypothesis requiring backend access, a staging build, or additional evidence.
Limitation	A constraint that reduced coverage or prevented a definitive conclusion.

01

Executive Summary

A concise view of validated risk, material uncertainty, and the actions that matter most.

OVERALL RISK

HIGH — driven by confirmed logging of OTP/SMS content in a production code path. No account takeover, OTP bypass, or OTP redirection was demonstrated during the supplied testing window.

Assessment outcome

The Max it Tunisie Android application was assessed using static analysis, manual reverse engineering, runtime observation, network interception attempts, and dynamic instrumentation. Automated results were manually reviewed so that scanner warnings were not automatically promoted to vulnerabilities.

Three issues are retained as validated findings: exposure of OTP/SMS content through application logging, failed Android App Link verification for declared domains, and a manifest configuration that permits cleartext traffic. The OTP logging issue is the most urgent because authentication secrets enter a loggable client-side code path.

Assessment disposition

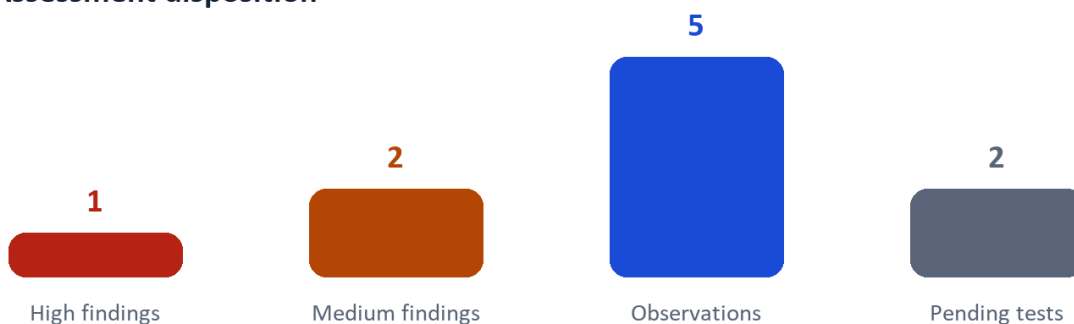


Figure 1 — Findings and test disposition. Observations and pending tests are intentionally excluded from the validated vulnerability count.

Key decisions for management

Priority	Decision
P0	Remove OTP and full SMS body logging; review telemetry sinks
P1	Repair or remove unverified App Link domains
P1	Disable cleartext traffic globally; allow exceptions only by named domain
P1	Complete backend-assisted OTP abuse and MSISDN binding tests

What was not proven

- No account takeover, OTP bypass, or OTP redirection was demonstrated.
- No repeated successful OTP backend replay was captured; server-side rate limiting remains unverified.
- No sensitive production request was observed over HTTP.
- No exploitable WebView or mini-app bridge method was demonstrated.
- No evidence showed API keys being unrestricted or OTP/PII being transmitted to telemetry.

02

Risk Register

ID	Title	Class	Severity
F-01	OTP/SMS content logged and passed to Flutter	Validated	High
F-02	Android App Link verification failures	Validated	Medium
F-03	Cleartext traffic permitted by manifest	Validated	Medium
O-01	WebView and TMF/Tencent mini-app attack surface	Observation	Contextual
M-07	Hardcoded service configuration values	Observation	Medium if unrestricted
M-08	Broad sensitive permissions and privacy exposure	Observation	Low / Medium
O-04	Third-party telemetry exposure pathway	Observation	Contextual
O-05	Legacy cryptographic markers	Observation	Contextual
M-02	Potential OTP abuse / insufficient rate limiting	Pending test	High if confirmed
M-03	Potential MSISDN modification / OTP redirection	Pending test	Critical if confirmed

03

Engagement Overview

Scope, target identity, test boundaries, and evidence constraints.

Target profile

Application	Max it Tunisie
APK	MaxIt.apk
Package name	com.orange.myorange.otn
Main activity	com.samea.super_app.MainActivity
Version	10.0.6
Target SDK	36
Minimum SDK	24
MobSF result	40/100 · Medium Risk · Grade B
APK SHA-256	301c9e7587e72582e599abc786459e657467208f8b0655be41a63bf548f0b47b

Assessment objectives

- Identify security weaknesses in the Android client and its platform integrations.
- Validate high-value automated findings through manual review and runtime evidence.
- Examine authentication-sensitive flows, especially OTP handling and MSISDN binding.
- Assess network transport, deep links, WebViews, mini-app components, permissions, and telemetry exposure.
- Distinguish confirmed vulnerabilities from hardening items and untested hypotheses.

In-scope testing areas

Domain	Coverage
Authentication	OTP/SMS receiver behavior, client-side handling, rate-limit and MSISDN hypotheses
Platform	Manifest, exported components, permissions, Android App Links, WebView behavior
Network	Proxy configuration, TLS interception attempts, transport configuration, endpoint discovery
Code	Reverse engineering, hardcoded configuration, cryptographic markers, logging
Privacy	Sensitive logging, telemetry pathways, third-party SDK presence, data exposure
Resilience	Frida/Gadget instrumentation and runtime resistance observations

04

Methodology and Risk Model

Evidence-led mobile testing aligned to current industry control domains.

Testing approach

1. Inventory the APK, package metadata, components, permissions, SDKs, and declared network/deep-link configuration.
2. Use automated static analysis to establish leads, then manually validate high-impact results in JADX and APKTool.
3. Observe runtime behavior through ADB/logcat, Android emulator and physical-device execution.
4. Attempt HTTPS interception and request modification through Burp Suite; investigate trust and custom-networking barriers.
5. Use Frida/Frida Gadget for runtime instrumentation and value-flow observation where feasible.
6. Classify each result as validated, contextual, pending, excluded, or a testing limitation based on the available evidence.

Tools and evidence sources

Tool / source	Purpose
MobSF	Initial static analysis, manifest review, permissions, trackers, and hotspots
JADX / APKTool	Decompiled code, manifest, resource, and component validation
ADB / logcat	Runtime process, plugin, receiver, and service observations
Burp Suite	Proxying, certificate trust validation, traffic observation, request-modification attempts
Frida / Gadget	Dynamic instrumentation and runtime value/network observation
Android Studio emulator / physical device	Controlled and real-device execution
curl / DNS checks	Domain reachability and Digital Asset Links availability

05

Validated Findings

Weaknesses supported by sufficient technical evidence are included in this section.

F-01 HIGH	OTP/SMS Content Logged and Passed to Flutter
Status	Validated
Confidence	High
Affected component	SmsBroadcastReceiver and Flutter MethodChannel otp_pin_field
Evidence	JADX code review; ADB/logcat receiver initialization

DESCRIPTION

The Google SMS Retriever broadcast receiver reads the full SMS message, writes that value to Android debug logs, and passes it to the Flutter layer. OTP values are authentication secrets and should not be placed in application logs or propagated as a full SMS body.

TECHNICAL EVIDENCE

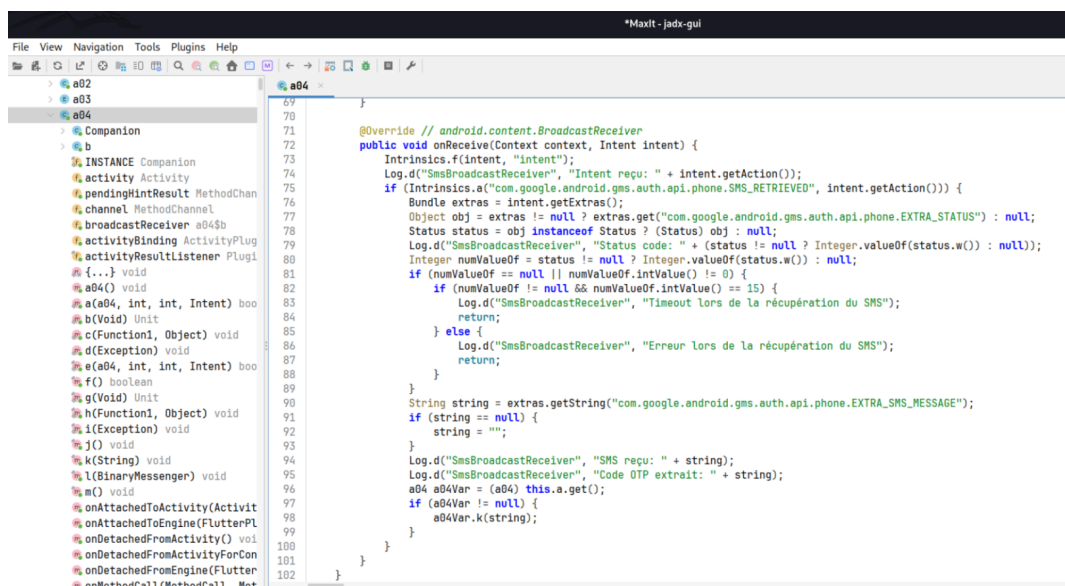


Figure 2 — Decompiled receiver reads, logs, and forwards the SMS message string.

JADX code review showed that the broadcast receiver listens for:

com.google.android.gms.auth.api.phone.SMS_RETRIEVED

It then retrieves the SMS content from:

com.google.android.gms.auth.api.phone.EXTRA_SMS_MESSAGE

The code logs the value as both the received SMS and extracted OTP:

Log.d("SmsBroadcastReceiver", "SMS reçu: " + string)

Log.d("SmsBroadcastReceiver", "Code OTP extrait: " + string)

The receiver listens for SMS_RETRIEVED, reads EXTRA_SMS_MESSAGE, logs the same string as the received SMS and extracted OTP, and passes it through `a04Var.k(string)`. The Flutter bridge invokes `smsCode` on `otp_pin_field`. Runtime logs confirmed receiver initialization; the supplied evidence does not show a live OTP value in logcat.

SECURITY IMPACT

- OTP or full SMS content may reach privileged log readers, compromised devices, support collection, crash pipelines, or runtime instrumentation.
- Forwarding the message broadens secret handling; combined with another control failure, OTP disclosure can contribute to account takeover.

REMEDIATION

1. Remove every log statement that can contain OTP values or SMS bodies from production code.
2. Parse only the expected OTP with a strict, bounded pattern and pass only that value to the minimum required component.
3. Do not forward the full SMS body to Flutter or any analytics, crash-reporting, support, or telemetry SDK.
4. Add a centralized redaction policy for OTPs, MSISDNs, tokens, authorization headers, and personal data.
5. Add CI checks that fail release builds when sensitive-field names or values reach logging APIs.

F-02 MEDIUM	Android App Link Verification Failures
Status	Validated
Confidence	High
Affected component	Browsable intent filters in <code>com.samea.super_app.MainActivity</code>
Evidence	Manifest review; DNS/curl result; adb pm get-app-links output

DESCRIPTION

The application declares multiple HTTPS App Link hosts, but supplied command output shows failed or unverified domain association states. At least `maxitci.page.link` did not resolve during testing, preventing retrieval of its Digital Asset Links file.

TECHNICAL EVIDENCE

```
(harzalls@kali)-[~]
$ curl -i https://maxitci.page.link/.well-known/assetlinks.json
curl: (6) Could not resolve host: maxitci.page.link

(harzalls@kali)-[~]
$ adb shell pm get-app-links com.orange.myorange.otn
com.orange.myorange.otn:
ID: 2479ae3e-d670-47e6-bd25-752b0aeb2796
Signatures: [10:04:4D:5E:16:C1:4F:64:B1:4F:E4:4D:3A:E0:37:56:C8:AE:56:2E:BE:03:5D:1A:F2:B3:1A:39:4F:68:B5:E5]
Domain verification state:
oci.page.link: 1024
orangecm.page.link: 1024
www.samea.com: 1024
maxitci.page.link: 1024
samea.com.link: 1024
maxitocipp.com.link: 1024
samea.page.link: 1024
maxitocmpp.com.link: 1024
multi.app.orange-money.com: 1024
maxitocmint.com.link: 1024
maxittn.page.link: 1024
maxit-link.com: 1024
maxitorangecm.page.link: 1024
maxit.page.link: 1024
maxit.com.link: 1024
maxitociint.com.link: 1024

(harzalls@kali)-[~]
$ curl -i "https://my-orange-tunisie.firebaseio.com/.json?shallow=true"
curl: (6) Could not resolve host: my-orange-tunisie.firebaseio.com
```

Figure 3 — DNS failure and Android domain-verification output for declared hosts.

Declared hosts included maxit.page.link, samea.page.link, maxitn.page.link, oci.page.link, maxitci.page.link, maxitorangecm.page.link, www.samea.com, maxit-link.com, and multi.app.orange-money.com. The evidence shows state 1024 for multiple entries and DNS failure for maxitci.page.link.

SECURITY IMPACT

Unverified links do not provide the trusted domain-to-application association expected from Android App Links. If affected URLs carry authentication, payment, onboarding, magic-link, or account-recovery data, a competing application may be able to register for similar links or users may be routed through a disambiguation flow. The report does not demonstrate interception of a sensitive token, so severity remains Medium.

REMEDIATION

1. Inventory every declared host and remove obsolete, unowned, or non-resolving domains from the manifest.
2. Publish a valid /.well-known/assetlinks.json over HTTPS on every active domain, without redirects, using the production package and signing-certificate SHA-256 fingerprint.
3. Ensure applicable intent filters request automatic verification and minimize broad path matching.
4. Treat all incoming URI data as untrusted; enforce strict allowlists for scheme, host, path, and parameters.
5. Re-verify links on supported Android versions and test sensitive flows end-to-end.

F-03 MEDIUM	Cleartext Traffic Permitted by the Manifest
Status	Validated configuration weakness
Confidence	High
Affected component	Android application manifest / network security policy
Evidence	MobSF and APKTool manifest review

DESCRIPTION

The application manifest permits cleartext network traffic through android:usesCleartextTraffic="true". This broad policy allows compatible platform and application networking components to use plaintext protocols unless separately restricted.

SECURITY IMPACT

If any sensitive workflow uses HTTP, an on-path attacker could read or alter requests and responses, steal identifiers or tokens, or inject content. No sensitive production request over HTTP was demonstrated in the supplied evidence, so this is reported as a confirmed configuration weakness with conditional downstream impact.

REMEDIATION

1. Set android:usesCleartextTraffic="false" in the release manifest.
2. Use a Network Security Configuration that denies cleartext by default and defines narrowly scoped exceptions only when formally approved.
3. Remove development and test endpoints from production builds.
4. Exercise every authenticated and unauthenticated workflow while monitoring traffic to confirm that no production endpoint uses HTTP.
5. Add a release-pipeline check for cleartext-enabled manifests and non-HTTPS endpoint strings.

Security Observations

Material hardening and assurance items that are not presented as validated vulnerabilities.

O-01 OBSERVATION

WebView and TMF/Tencent Mini-App Attack Surface

The application integrates WebView and TMF/Tencent mini-app components. This creates a broad attack surface because mini-app frameworks often expose JavaScript bridges, dynamic content loading, file APIs, media APIs, and network APIs.

MobSF identified multiple app components and exported components:

Activities: 44

Services: 21

Receivers: 19

Providers: 13

Exported Activities: 2

Exported Services: 1

Exported Receivers: 3

Runtime logs confirmed TMF mini-app process registration under the Orange package, including mini-app and mini-game processes:

com.orange.myorange.otn:mini5

com.orange.myorange.otn:mini6

with TMF mini-app launcher activity and receiver registration.

Static analysis also identified WebView and WebMessage-related components, including WebMessage listener/channel strings and TMF/Tencent networking/API logic.

RECOMMENDED ASSURANCE ACTIONS

- Maintain explicit origin/app-ID allowlists and audit every JavaScript/native bridge method.
- Disable WebView debugging in release builds and keep the SDK current.

M-07 OBSERVATION

Hardcoded Service Configuration Values

Potential severity	Medium if unrestricted
Status	Confirmed static; exploitability depends on key restrictions
Category	Secret and configuration management
Evidence sources	MobSF; strings/grep searches

DESCRIPTION

Static analysis identified embedded service configuration values and possible API keys. These values are not automatically critical because many mobile SDKs require public client-side configuration. The risk depends on whether each key is properly restricted and whether any embedded value grants privileged backend access.

TECHNICAL EVIDENCE AND PROVENANCE

- MobSF flagged files that may contain hardcoded sensitive information, including Batch, Huawei, InAppWebView, and TMF/Tencent-related classes.
- The scanner mapped these leads to cleartext storage of sensitive information and mobile reverse-engineering risk; this is a lead, not proof that a usable secret was exposed.

- Independent strings/grep searches identified Firebase, Crashlytics, Google API, and Batch-related configuration strings in the APK.
- No supplied test demonstrated that a discovered key was unrestricted, reusable outside the official application, or authorized for a privileged backend operation.

BUSINESS IMPACT

If any embedded API key is unrestricted, an attacker could:

- reuse the key outside the official application and consume API quotas;
- interact with APIs that were unintentionally enabled for the key;
- abuse Firebase services where backend security rules are weak;
- discover internal project metadata or misuse push and analytics configuration.

SEVERITY JUSTIFICATION

This item is Medium if an unrestricted key can be abused. If all values are public client identifiers and every key is restricted by package name, signing certificate, API scope, and backend rules, the residual risk is informational.

REMEDIATION

1. Restrict Android API keys by package name and SHA-256 signing-certificate fingerprint.
2. Restrict each key to the minimum required APIs and environments.
3. Review Firebase Realtime Database and Firestore rules independently of client configuration.
4. Remove test keys and configuration from production builds.
5. Rotate unrestricted, exposed, or unused keys and monitor subsequent use.
6. Never embed backend secrets or privileged service credentials in the APK.

M-08 DETAILED OBSERVATION

M-08
OBSERVATION

Broad Sensitive Permissions and Privacy Exposure

Severity	Low / Medium
Status	Confirmed, contextual
Category	Privacy and least privilege
Evidence source	Android merged manifest and MobSF permission analysis
Description	The application requests several Android dangerous permissions. Permissions are not vulnerabilities by themselves, but unnecessary or overly broad grants increase privacy exposure and post-compromise impact. Risk also increases if embedded WebView, mini-app, or third-party SDK components can reach sensitive capabilities indirectly.

TECHNICAL EVIDENCE

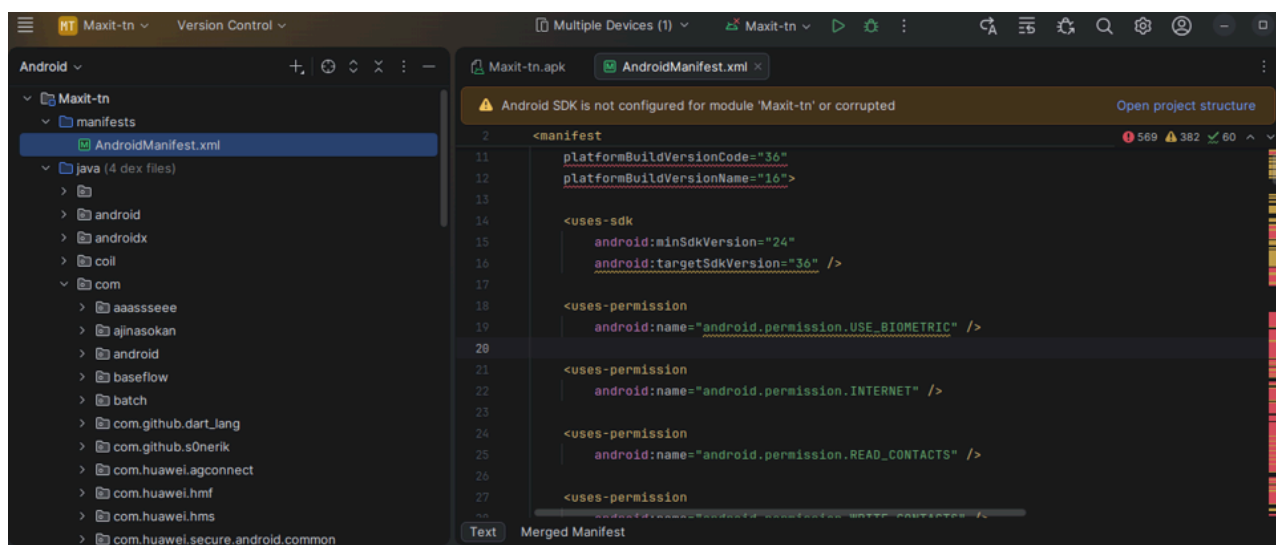


Figure 4 — Android Studio merged-manifest review showing declared permissions, including USE_BIOMETRIC, INTERNET, and READ_CONTACTS.

PERMISSION	STATUS	INFO	DESCRIPTION
android.permission.RECORD_AUDIO	dangerous	record audio	Allows application to access the audio record path.
android.permission.CAMERA	dangerous	take pictures and videos	Allows application to take pictures and videos with the camera. This allows the application to collect images that the camera is seeing at any time.
android.permission.READ_PHONE_STATE	dangerous	read phone state and identity	Allows the application to access the phone features of the device. An application with this permission can determine the phone number and serial number of this phone, whether a call is active, the number that call is connected to and so on.
android.permission.BLUETOOTH_ADVERTISE	dangerous	required to advertise to nearby Bluetooth devices.	Required to be able to advertise to nearby Bluetooth devices.
android.permission.BLUETOOTH_CONNECT	dangerous	necessary for connecting to paired Bluetooth devices.	Required to be able to connect to paired Bluetooth devices.
android.permission.BLUETOOTH_SCAN	dangerous	required for discovering and pairing Bluetooth devices.	Required to be able to discover and pair nearby Bluetooth devices.

Figure 5 — MobSF classifies RECORD_AUDIO, CAMERA, READ_PHONE_STATE, and Bluetooth advertise/connect/scan permissions as dangerous.

The complete permission inventory recorded by MobSF includes:

- READ_CONTACTS and WRITE_CONTACTS
- READ_EXTERNAL_STORAGE and WRITE_EXTERNAL_STORAGE
- ACCESS_FINE_LOCATION and ACCESS_COARSE_LOCATION
- CAMERA and RECORD_AUDIO
- READ_PHONE_STATE
- BLUETOOTH_SCAN, BLUETOOTH_ADVERTISE, and BLUETOOTH_CONNECT
- READ_CALENDAR and WRITE_CALENDAR

EVIDENCE
INTERPRETATION

The merged manifest proves that permissions are declared; MobSF proves Android classifies the displayed permissions as dangerous. Neither artifact alone proves that every permission is requested at runtime, used without a legitimate feature need, or exposed to embedded content.

BUSINESS IMPACT

If unnecessary permissions are granted, the application may increase exposure of contact data, location data, calendar data, phone state, camera and audio capture, local files, and nearby Bluetooth-device information. This deserves special review because WebView and mini-app content must not inherit sensitive native capabilities without explicit authorization.

SEVERITY JUSTIFICATION

The issue is rated Low / Medium because legitimate business functionality may justify many permissions. Severity increases only where a permission is unnecessary, over-broad, requested prematurely, or reachable from an untrusted component.

REMEDIATION

1. Apply least privilege and document the business feature that requires each permission.
2. Remove unused permissions, including permissions inherited from third-party libraries where feasible.
3. Request dangerous permissions at runtime, at the point of need, and only after clear user context.
4. Provide concise user-facing explanations for sensitive access.
5. Prevent WebView and mini-app content from reaching dangerous native permissions unless explicitly authorized and origin-bound.
6. Review and constrain third-party SDK access to sensitive data and platform capabilities.

O-04
OBSERVATION

Third-party telemetry exposure pathway

The application integrates several third-party SDKs, including analytics, crash reporting, push, and attribution components. These are not vulnerabilities by themselves, but they become important when sensitive data such as OTP, MSISDN, tokens, or personal information is logged or sent to telemetry.

MobSF detected 5 trackers out of 432, and the report identified tracker-related components.

Runtime logs showed Batch and Firebase Messaging activity, including Batch service errors and Firebase token retrieval failures during emulator testing.

RECOMMENDED ASSURANCE ACTIONS

- Inspect OTP/auth telemetry and apply centralized redaction before third-party SDKs.
- Document purpose, retention, destination, and lawful basis for personal-data telemetry.

O-05
OBSERVATION

Weak Crypto / Legacy Hashing Patterns

Static analysis identified weak or legacy cryptographic patterns, including MD5 usage and legacy signing/certificate metadata. These are not automatically exploitable, because the risk depends on how the algorithms are used.

Weak crypto becomes security-relevant if used for:

- password hashing
- token generation
- signature validation
- integrity checks
- encryption of sensitive local data
- authentication decisions

If used only for cache keys or non-security identifiers, impact is low.

RECOMMENDED ASSURANCE ACTIONS

- Replace MD5 with SHA-256 or stronger where security-relevant.
- Use HMAC-SHA256 for keyed integrity.
- Avoid CBC without integrity protection.
- Review all crypto usage manually.

06

Pending Validation

High-impact hypotheses that require controlled backend-assisted testing before they can be called vulnerabilities.

**M-02
PENDING****Potential OTP Abuse / Insufficient Rate Limiting**

Potential severity	High if confirmed
Status	Pending validation
Category	Authentication abuse / SMS abuse
Evidence sources	JADX; ADB/logcat; Burp Suite attempts; manual testing
Reason pending	OTP send request was not fully captured and replayed

DESCRIPTION

The OTP flow was confirmed to exist and execute at runtime. However, the exact backend request responsible for sending OTP messages was not fully captured, modified, and replayed in Burp Suite. Insufficient rate limiting therefore cannot be reported as a confirmed vulnerability. It remains a high-priority pending risk because weak server-side throttling can enable material SMS abuse.

TECHNICAL EVIDENCE AND PROOF TRAIL

- JADX/static searches found OTP and SMS-related symbols: SmsBroadcastReceiver, SmsRetriever, smsCode, smsCodeRegexPattern, and sms_retrieve.
- ADB/logcat confirmed during runtime that the OTP listener was started, proving the client-side OTP receive path was active.
- Burp Suite was configured to intercept emulator and physical-device traffic, but the production OTP API request could not be fully captured and modified because of TLS/certificate trust issues and likely custom networking behavior.
- Manual testing did not demonstrate repeated successful backend OTP generation or a bypass of a server-enforced cooldown.

**EVIDENCE
BOUNDARY**

The evidence proves that the OTP flow and listener exist. It does not prove that the backend lacks rate limiting. Confirmation requires a captured send-OTP request, controlled replay, and server/SMS delivery evidence.

BUSINESS IMPACT

If rate limiting is weak, an attacker could:

- repeatedly trigger OTP messages and cause SMS flooding;
- increase SMS delivery costs and exhaust OTP quotas;
- harass users and degrade authentication availability;
- abuse telecom infrastructure at scale.

SEVERITY JUSTIFICATION

This issue would be High if confirmed because OTP abuse can create direct financial, operational, and user impact. It remains pending because no repeated successful backend replay or resulting SMS delivery was demonstrated.

REMEDIATION

1. Enforce server-side rate limiting per MSISDN, account, IP address, device identifier, and session.
2. Add an enforced cooldown between OTP requests.
3. Return HTTP 429 Too Many Requests, or an equivalent consistent response, when limits are exceeded.
4. Monitor and alert on abnormal OTP request volumes and delivery patterns.
5. Block or challenge repeated requests from suspicious clients and correlated sources.
6. Do not rely on mobile client-side restrictions as the security control.

VALIDATION PLAN

1. Capture the OTP-generation request and identify server-controlled account, MSISDN, session, device, and challenge identifiers.
2. Repeat requests at increasing frequency from the same and varied IPs/devices while respecting an approved test ceiling.
3. Measure cooldown, per-dimension limits, response semantics, OTP invalidation behavior, and monitoring/alerting.
4. Confirm that controls are enforced server-side and cannot be bypassed through client changes, header variation, or parallel sessions.

SECURE ACCEPTANCE CRITERIA

- Rate limits apply per MSISDN/account and include IP, device, session, and behavioral controls.
- The service returns a consistent throttling response and does not send additional SMS messages during cooldown.
- Repeated abuse is logged, alerted, and does not reveal whether an account exists.

**M-03
PENDING****Potential MSISDN Modification / OTP Redirection**

Potential severity	Critical if confirmed
Status	Pending validation
Category	Authentication logic flaw
Evidence sources	JADX; grep/strings; Frida attempts; Burp Suite attempts
Reason pending	No production OTP request containing the MSISDN was successfully modified

The assessment investigated whether the phone number/MSISDN used in the OTP flow could be modified before the request reached the backend. Static searches found MSISDN-related and SMS-request references, but the real production backend request containing the MSISDN was not fully captured. Some discovered methods appeared to belong to embedded TMF/Tencent mini-app components rather than the Orange production OTP flow. No OTP redirection was demonstrated, so this remains a pending risk.

TECHNICAL EVIDENCE AND PROOF TRAIL

JADX and strings/grep searches examined the following terms and method names:

- msisdn, getMsisdn, and setMsisdn
- sendSmsCodeRequest and sendVerifySmsCodeRequest
- phone, sms, and otp
- Static analysis also found SMS Retriever, SMS broadcast receiver, and related API strings.
- Several promising SMS methods were associated with TMF/Tencent embedded components; attribution to the production Orange OTP workflow was not established.
- Frida/Gadget instrumentation attempted to observe and change values passed through Java methods, Flutter MethodChannels, and networking APIs.
- Burp Suite interception was attempted, but no production request with a mutable MSISDN field was captured and no successful MSISDN tampering or OTP redirection occurred.

**EVIDENCE
BOUNDARY**

String and method-name matches establish candidate code paths only. A critical finding requires proof that the production backend accepts a changed destination and sends or validates the OTP for the attacker-selected MSISDN.

BUSINESS IMPACT

If the backend accepts a client-modified MSISDN, an attacker could potentially:

- redirect OTP delivery to another number;
- bypass phone-ownership verification;
- bind a victim account to an attacker-controlled phone number;
- take over affected accounts.

SEVERITY JUSTIFICATION

This issue would be Critical if confirmed because it could directly break the authentication and account-binding model. It is not marked confirmed because no successful OTP redirection was demonstrated.

REMEDIATION

1. Bind OTP generation and verification server-side to the same MSISDN, account, device policy, and session.
2. Do not trust a client-supplied MSISDN as authoritative identity or destination state.
3. Reject verification if the MSISDN or bound subject changes between OTP generation and validation.
4. Store OTP challenge state server-side, make it single-use, and expire it quickly.
5. Validate this scenario using backend logs or an interceptable staging build and approved test identities.

VALIDATION PLAN

1. Record the OTP generation transaction and its server-issued challenge identifier.
2. Attempt to change the MSISDN, account, device, and session independently between generation and verification.
3. Attempt parallel challenges and cross-account/cross-session verification using only approved test identities.
4. Use backend logs to confirm that the server binds OTP challenge state to the intended subject and destination.

SECURE ACCEPTANCE CRITERIA

- The server ignores or rejects unauthorized client-side changes to MSISDN and subject identity.
- Generation and verification remain bound to the same account, MSISDN, session, device policy, and single-use challenge.
- A challenge cannot be replayed, swapped across users, or validated after a material context change.

07

Assessment Limitation

L-01
INFORMATIONAL

TLS Interception and Dynamic Coverage Limitation

Burp Suite was configured for emulator and physical-device testing, and some browser/background traffic was observable. The OTP backend request was not fully captured or modified because of certificate trust issues and possible custom networking behavior. Frida/Gadget instrumentation was available, but it did not close the request-level coverage gap during the supplied testing window.

Why this matters

This is not a vulnerability. It explains why backend-dependent authentication risks remain pending and why the absence of an observed exploit must not be interpreted as proof that server-side controls are effective.

Recommended enablement for the retest

- Provide a staging build that trusts an internal test CA or an approved test-only mechanism for observing traffic.
- Provide API documentation, backend request correlation IDs, and logs for OTP generation and verification.
- Provide dedicated test accounts and MSISDNs with pre-approved abuse thresholds.
- Document whether certificate pinning or custom TLS validation is expected in the release build.

08

Remediation Roadmap

Action	Validation evidence
Remove OTP/SMS logging and full-message forwarding; deploy telemetry redaction	Clean logcat and telemetry captures across OTP scenarios
Repair/remove App Link domains and verify production associations	Successful Android link verification for every active host
Disable cleartext traffic and enforce domain-scoped network policy	Release manifest, config review, and traffic test
Execute backend-assisted OTP throttling and MSISDN binding tests	Approved test log, server evidence, and closed M-02/M-03 decisions
Audit WebView/mini-app bridges, origins, permissions, and debugging	Bridge inventory, allowlists, release-build verification
Review API-key restrictions, telemetry, dangerous permissions, and crypto use	Configuration exports, data-flow evidence, permission matrix
Add security gates to CI/CD and schedule regression testing	Pipeline controls and recurring MASVS-aligned test evidence

09

Conclusion

The Max it Tunisie Android application presents a medium-to-high mobile security risk profile.

The strongest confirmed vulnerability is the exposure of OTP/SMS content through client-side logging and forwarding to Flutter. This finding is supported by JADX code review and ADB/logcat runtime evidence, not just automated scanning.

The next most important confirmed issues are broken Android App Link verification, cleartext traffic being enabled, WebView/mini-app attack surface, hardcoded service configuration values, broad sensitive permissions, and third-party telemetry exposure risk.

Several MobSF findings were filtered out or downgraded because they are not real vulnerabilities by themselves. This includes generic dangerous permissions, anti-VM checks, trackers, raw SQL warnings, and some protected exported components. They should be reviewed, but they should not be exaggerated without exploit evidence.

The most important pending risks are OTP rate limiting and MSISDN modification. If either is confirmed through backend-assisted testing or successful request interception, the severity could become High or Critical, especially if they allow OTP flooding or account takeover.